



Resumo Completo da Matéria — Paradigmas de Programação (PP)

Nota de Orientação: Este resumo foi elaborado de acordo com os conteúdos lecionados na UC de Paradigmas de Programação (ESTG/P.PORTO - LEI & LSIRC). **Regra Obrigatória dos Exames:** É estritamente **PROIBIDO** o uso da biblioteca `java.util` (`ArrayList`, `List`, `Arrays`, `Scanner`, etc.). Todas as coleções são geridas manualmente através de **arrays nativos** (`[]`) e contadores de elementos.

1. Módulo 1: Fundamentos da Linguagem Java (APP01 & APP02)

1.1. Tipos Primitivos e Valores por Defeito

Em Java existem 8 tipos primitivos divididos por categorias:

Tipo Primitivo	Tamanho	Valor por Defeito	Gama / Descrição
<code>boolean</code>	1 bit (lógico)	<code>false</code>	<code>true</code> ou <code>false</code>
<code>byte</code>	8 bits	<code>0</code>	-128 a 127
<code>short</code>	16 bits	<code>0</code>	-32.768 a 32.767
<code>int</code>	32 bits	<code>0</code>	-2.147.483.648 a 2.147.483.647
<code>long</code>	64 bits	<code>0L</code>	Inteiros longos (sufixo <code>L</code> ou <code>l</code>)
<code>float</code>	32 bits	<code>0.0f</code>	Vírgula flutuante IEEE 754 (sufixo <code>F</code> ou <code>f</code>)
<code>double</code>	64 bits	<code>0.0d</code>	Vírgula flutuante de dupla precisão (padrão)
<code>char</code>	16 bits	<code>'\u0000'</code>	Caracter Unicode (aspas simples, ex: <code>'A'</code>)

Nota de Exame: As variáveis locais de um método **NÃO** têm valor por defeito e têm de ser explicitamente inicializadas antes do primeiro uso, sob pena de erro de compilação. Apenas os atributos de classe/instância recebem os valores por defeito da tabela acima.

1.2. Literais e Constantes

- **Constantes:** Declaradas com a palavra reservada `final`. O seu valor não pode ser alterado após a inicialização.

```
public static final double PI = 3.14159;
final int MAX_ITEMS = 100;
```

1.3. Operadores e Controlo de Fluxo

- **Operadores Relacionais e Lógicos:** `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&` (AND curto-circuito), `||` (OR curto-circuito), `!` (NOT).
- **Estruturas Condicionais:** `if-else`, `switch-case` (suporta `byte`, `short`, `char`, `int`, `String` e `enum`).
- **Ciclos de Repetição:** `for`, `while`, `do-while`, `for-each` (em arrays).

```
// Ciclo for-each sobre array nativo
int[] valores = {10, 20, 30};
for (int val : valores) {
    System.out.println(val);
}
```

2. Módulo 2: Introdução à Orientação a Objetos (APP03)

2.1. Classe vs Objeto

- **Classe:** O molde/modelo abstrato que define a estrutura (atributos de estado) e as ações/comportamentos (métodos).
- **Objeto:** Uma instância concreta da classe criada em memória (na *Heap*) através do operador `new`.

2.2. Encapsulamento, Atributos e Métodos

O encapsulamento protege os dados internos de modificações indevidas. Atributos devem ser `private`, sendo expostos através de métodos seletores (*getters*) e modificadores (*setters*).

```
public class Contentor {
    private String codigo;
    private double capacidadeMax;
    private double ocupacaoAtual;

    public Contentor(String codigo, double capacidadeMax) {
        this.codigo = codigo;
        this.capacidadeMax = capacidadeMax;
        this.ocupacaoAtual = 0.0;
    }

    public String getCodigo() {
        return this.codigo;
    }
}
```

```

    public double getCapacidadeMax() {
        return this.capacidadeMax;
    }

    public double getOcupacaoAtual() {
        return this.ocupacaoAtual;
    }

    public void setOcupacaoAtual(double ocupacaoAtual) {
        if (ocupacaoAtual >= 0 && ocupacaoAtual <= this.capacidadeMax) {
            this.ocupacaoAtual = ocupacaoAtual;
        }
    }
}

```

2.3. A Palavra-Chave `this`

Utilizada dentro de métodos de instância para:

1. Referenciar o próprio objeto corrente (`this.atributo`).
2. Resolver ambiguidades entre nomes de atributos e parâmetros do construtor/método.
3. Chamar outro construtor da mesma classe (`this(...)`), devendo ser obrigatoriamente a primeira instrução do construtor.

3. Módulo 3: Passagem de Parâmetros e Gestão Manual de Arrays (APP04)

3.1. Passagem de Parâmetros em Java: Sempre por Valor!

Java realiza a passagem de parâmetros **EXCLUSIVAMENTE POR VALOR**:

1. **Tipos Primitivos:** É passada uma **cópia do valor**. Modificações ao parâmetro dentro do método **não afetam** a variável original.
2. **Tipos de Referência (Objetos e Arrays):** É passada uma **cópia do valor da referência (endereço na Heap)**.
 - Alterar o estado do objeto apontado pela referência (ex: `obj.setCapacidade(...)` ou `arr[0] = 5`) **modifica o objeto original**.
 - Reatribuir a variável de referência dentro do método (ex: `obj = new Contentor(...)`) **NÃO altera a referência original** fora do método.

3.2. Gestão Manual de Coleções com Arrays Nativos (SEM `java.util`)

Como o uso de `java.util` é proibido, a gestão de múltiplos objetos é feita combinando um **array nativo fixo** com uma **variável contadora de tamanho** (`count`).

Padrão de Adicionar Elemento:

```

public class AidBox {
    private Container[] containers;
    private int count;

    public AidBox(int maxContainers) {
        this.containers = new Container[maxContainers];
        this.count = 0;
    }

    public boolean addContainer(Container c) {
        if (c == null || this.count >= this.containers.length) {
            return false; // Array cheio ou elemento nulo
        }
        this.containers[this.count] = c;
        this.count++;
        return true;
    }
}

```

Padrão de Redimensionamento Dinâmico (Expandir Array):

```

private void resize() {
    Container[] temp = new Container[this.containers.length * 2];
    System.arraycopy(this.containers, 0, temp, 0, this.count);
    this.containers = temp;
}

```

Padrão de Remoção Compactando o Array:

```

public boolean removeContainer(String codigo) {
    for (int i = 0; i < this.count; i++) {
        if (this.containers[i].getCodigo().equals(codigo)) {
            // Deslocar os elementos seguintes para a esquerda
            for (int j = i; j < this.count - 1; j++) {
                this.containers[j] = this.containers[j + 1];
            }
            this.containers[this.count - 1] = null; // Limpar a última posição
            this.count--;
            return true;
        }
    }
    return false;
}

```

4. Módulo 4: Herança, Polimorfismo e Casting (APP05)

4.1. Herança (`extends`) e `super`

A herança estabelece uma relação lógica "**É UM**" (*is-a*). Uma subclasse herda todos os membros não privados da superclasse.

- `super(...)` : Chama o construtor da superclasse (deve ser a primeira linha do construtor da subclasse).
- `super.metodo()` : Invoca a versão do método definida na superclasse.

```
public class Veiculo {
    private String codigo;
    private double capacidadeMax;

    public Veiculo(String codigo, double capacidadeMax) {
        this.codigo = codigo;
        this.capacidadeMax = capacidadeMax;
    }

    public String getCodigo() { return this.codigo; }
    public double getCapacidadeMax() { return this.capacidadeMax; }
}

public class VeiculoRefrigerado extends Veiculo {
    private double maxQuilometros;

    public VeiculoRefrigerado(String codigo, double capacidadeMax, double maxQuilometros) {
        super(codigo, capacidadeMax); // Invocação do construtor pai
        this.maxQuilometros = maxQuilometros;
    }

    public double getMaxQuilometros() {
        return this.maxQuilometros;
    }
}
```

4.2. Sobreposição (Overriding) vs Sobrecarga (Overloading)

- **Sobrecarga (Overloading)**: Mesma classe, métodos com o mesmo nome mas **assinaturas diferentes** (parâmetros diferentes). Decidida em tempo de compilação (*compile-time*).
- **Sobreposição (Overriding)**: Subclasse reinterpreta um método da superclasse com a **mesma assinatura exata**. Usa-se `@Override`. Decidida em tempo de execução (*runtime*) via ligação dinâmica (*dynamic binding*).

4.3. Polimorfismo e Conversão de Tipos (Casting)

O polimorfismo permite tratar objetos de subclasses como instâncias da superclasse.

- **Upcasting (Implícito / Seguro)**: Converter uma subclasse para superclasse. `Veiculo v = new VeiculoRefrigerado("V01", 500, 150);`
- **Downcasting (Explícito / Requer Validação)**: Converter uma superclasse para subclasse. Requer o operador `instanceof` para evitar a exceção `ClassCastException`.

```
if (v instanceof VeiculoRefrigerado) {
    VeiculoRefrigerado vr = (VeiculoRefrigerado) v; // Downcasting seguro
}
```

```
System.out.println("Km max: " + vr.getMaxQuilometros());  
}
```

5. Módulo 5: Abstração, Interfaces e Modificadores (APP06)

5.1. Classes e Métodos Abstratos (`abstract`)

- **Classe Abstrata:** Não pode ser instanciada diretamente (`new`). Serve como superclasse base. Pode ter estado (atributos), construtores, métodos concretos e métodos abstratos.
- **Método Abstrato:** Declarado sem corpo (`public abstract void processar();`). As subclasses concretas são **obrigadas** a implementar todos os métodos abstratos.

5.2. Interfaces (`interface` & `implements`)

- Define um **contrato puro de comportamento** (o que a classe faz, mas não como faz).
- Todos os métodos são implicitamente `public abstract` (exceto `default` e `static` das versões Java mais recentes).
- Todos os atributos declarados numa interface são implicitamente `public static final` (constantes).
- Uma classe pode implementar **múltiplas interfaces** (`implements A, B`), contornando a limitação da herança simples em Java.

5.3. Modificadores de Acesso

Modificador	Própria Classe	Mesmos Pacote	Subclasse (Outro Pacote)	Qualquer Local
<code>private</code>	✓	✗	✗	✗
<i>(default / package)</i>	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

5.4. Membros Estáticos (`static`)

- **Variáveis Estáticas:** Alocadas uma única vez na área de classes. Partilhadas por todas as instâncias da classe.
- **Métodos Estáticos:** Podem ser chamados sem instanciar a classe (ex: `Math.abs()`). **NÃO podem** aceder a membros de instância nem utilizar `this` ou `super` .

6. Módulo 6: Métodos da Classe `Object` (APP07)

Todas as classes em Java herdam implicitamente de `java.lang.Object` .

6.1. Identidade (`==`) vs Igualdade Lógica (`equals`)

- Operador `==` : Compara se duas referências apontam para o **mesmo endereço de memória**.
- Método `equals(Object obj)` : Compara a **igualdade lógica de valores/atributos** entre dois objetos.

6.2. Redefinição Correta dos Métodos `equals()` e `toString()`

Padrão obrigatório exigido pelos professores em exames:

```
public class VeiculoImpl implements Veiculo {
    private String code;
    private double maxCapacity;

    public VeiculoImpl(String code, double maxCapacity) {
        this.code = code;
        this.maxCapacity = maxCapacity;
    }

    @Override
    public String getCode() { return this.code; }

    @Override
    public double getMaxCapacity() { return this.maxCapacity; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Mesmo endereço na Heap
        if (obj == null || !(obj instanceof Veiculo)) return false; // Nulo ou tipo indevido

        Veiculo other = (Veiculo) obj;
        if (this.code == null) {
            return other.getCode() == null;
        }
        return this.code.equals(other.getCode());
    }

    @Override
    public String toString() {
        return "Veiculo[Codigo: " + this.code + ", CapMax: " + this.maxCapacity + "];"
    }
}
```

7. Módulo 7: Gestão de Exceções em Java (APP08)

7.1. Hierarquia de Exceções

- **Throwable** : Classe base de todos os erros e exceções.
 - **Error** : Erros graves do sistema/JVM (ex: `OutOfMemoryError`, `StackOverflowError`). Não devem ser capturados.
 - **Exception** : Erros aplicativos dos quais é possível/desejável recuperar.
 - **Checked Exceptions (Verificadas)**: Subclasses de `Exception` (excluindo `RuntimeException`). O compilador **obriga** a tratar com `try-catch` ou declarar com `throws`. Representam condições externas previsíveis.

- **Unchecked Exceptions (Não Verificadas):** Subclasses de `RuntimeException` (ex: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`). O compilador **não obriga** a declaração. Indicam bugs/erros de programação.

7.2. Bloco `try-catch-finally`

- `try` : Contém o código suscetível de lançar exceções.
- `catch` : Captura e trata exceções de tipos específicos.
- `finally` : Executado **SEMPRE**, quer ocorra exceção ou não (mesmo que haja `return` no `try` ou `catch`). Ideal para libertar recursos.

7.3. Exceções Personalizadas

```
public class RouteException extends Exception {
    public RouteException() {
        super();
    }

    public RouteException(String message) {
        super(message);
    }
}
```

8. Módulo 8: Enums e Tipos Enumerados (APP09)

Os Enums em Java são tipos estruturados com construtores e métodos.

```
public enum ItemType {
    PERISHABLE_FOOD("Comida Perecivel"),
    NON_PERISHABLE_FOOD("Comida Nao Perecivel"),
    CLOTHING("Vestuario"),
    MEDICINE("Medicamentos");

    private final String descricao;

    // Construtor obrigatoriamente privado
    private ItemType(String descricao) {
        this.descricao = descricao;
    }

    public String getDescricao() {
        return this.descricao;
    }
}
```

9. Módulo 9: Input/Output e Leitura de Ficheiros (APP10)

9.1. Byte Streams vs Character Streams

- **Byte Streams (`InputStream` / `OutputStream`):** Lêm/escrevem dados binários em bruto (8 bits per item). Usado para imagens, vídeos e ficheiros executáveis.
- **Character Streams (`Reader` / `Writer`):** Lêm/escrevem caracteres Unicode (16 bits). Gerem codificações de texto (ex: UTF-8) e são adequados para `.txt` , `.json` , `.xml` .

10. Módulo 10: Padrões de Modelação e Padrão Strategy (APP11 & APP12)

10.1. O Padrão de Desenho *Strategy*

O padrão *Strategy* encapsula uma família de algoritmos intermutáveis em classes separadas que implementam uma interface comum (`Strategy`). Permite alterar o algoritmo de cálculo (ex: geração de rotas de recolha) em tempo de execução sem alterar os clientes que o consomem.

```
public interface Strategy {
    Route[] generate(IIstitution inst, RouteValidator validator);
}
```

11. Tabela Resumo das Palavras Reservadas Críticas

Palavra-Chave	Contexto e Utilização
<code>abstract</code>	Declara classes que não podem ser instanciadas ou métodos sem corpo.
<code>extends</code>	Herança de classes (especialização).
<code>implements</code>	Implementação de contratos de interfaces.
<code>super</code>	Acesso ao construtor ou métodos da superclasse.
<code>this</code>	Referência à própria instância corrente.
<code>static</code>	Membros pertencentes à classe e partilhados por todas as instâncias.
<code>final</code>	Impede reatribuição de variáveis, sobreposição de métodos ou herança de classes.
<code>instanceof</code>	Teste de tipo dinâmico antes de efetuar downcasting.
<code>throws</code>	Declaração na assinatura do método das exceções verificadas lançadas.
<code>throw</code>	Lançamento explícito de uma instância de exceção.